

Odysseus AI Workspace

Vollständige Installations- und Konfigurationsdokumentation

Erstellt: Juni 2026 | Umgebung: Ubuntu/Linux Mint, Docker CE, Portainer, Nginx Proxy Manager

Host: **172.22.222.114** | Domain: **odysseus.stonecloud.at** | GPU: NVIDIA RTX PRO 2000
Blackwell

Was ist Odysseus?

Odysseus ist ein selbst gehosteter, KI-gestützter All-in-one Workspace. Er ist kein LLM selbst, sondern ein Orchestrator/Frontend, der lokale Modelle (Ollama), Cloud-APIs (OpenAI, DeepSeek, etc.) und zahlreiche integrierte Dienste (SearXNG, ChromaDB, ntfy) zusammenführt. Vergleichbar mit OpenWebUI, aber mit deutlich erweitertem Funktionsumfang: Chat, Agents, Deep Research, Dokumente, E-Mail, Notizen, Kalender, RAG, MCP-Server und mehr.

Inhaltsverzeichnis

- | | |
|----|--|
| 1 | Systemvoraussetzungen & Überblick |
| 2 | Ollama – Konfiguration für Docker-Zugriff |
| 3 | NVIDIA Container Toolkit installieren |
| 4 | Odysseus – Repository klonen & Branch wählen |
| 5 | Umgebungsdatei (.env) konfigurieren |
| 6 | Stack starten & GPU-Overlay aktivieren |
| 7 | Erster Login & Admin-Passwort |
| 8 | Nginx Proxy Manager – Reverse Proxy einrichten |
| 9 | Ollama in Odysseus einbinden |
| 10 | SearXNG (extern) einbinden |
| 11 | Portainer – Stack-Verwaltung |
| 12 | Referenz: Wichtige Ports & Netzwerke |

1. Systemvoraussetzungen & Überblick

Komponente	Wert / Version
Betriebssystem	Ubuntu 24.04 / Linux Mint (noble)
Docker CE	aktuell (jammy repo)
Portainer CE	aktuell
Nginx Proxy Manager	aktuell
Ollama	0.30.7 (läuft direkt am Host als systemd-Service)
GPU	NVIDIA RTX PRO 2000 Blackwell, 16 GB VRAM
NVIDIA Treiber	595.71.05 / CUDA 13.2
Host-IP (LAN)	172.22.222.114
Öffentliche Domain	odysseus.stonecloud.at
Odysseus Port	7000 (HTTP, intern)

Netzwerk-Architektur

Datenfluss:

Browser → <https://odysseus.stonecloud.at> → Nginx Proxy Manager (NPM) → <http://172.22.222.114:7000> → Odysseus Container

Odysseus Container → <http://host.docker.internal:11434/v1> → Ollama (Host, systemd)

Odysseus Container → <http://searxng:8080> → SearXNG Container (intern im odysseus_default Netz)

Odysseus Container → <http://chromadb:8000> → ChromaDB Container (intern)

2. Ollama – Konfiguration für Docker-Zugriff

Ollama läuft als systemd-Service direkt am Host. Standardmäßig lauscht es nur auf [127.0.0.1:11434](#) und ist damit für Docker-Container nicht erreichbar. Es muss auf allen Interfaces ([0.0.0.0](#)) gebunden werden.

1 systemd Drop-in Override anlegen

```
sudo mkdir -p /etc/systemd/system/ollama.service.d

sudo tee /etc/systemd/system/ollama.service.d/override.conf >/dev/null <<'EOF'
```

```
[Service]
Environment=
Environment="OLLAMA_HOST=0.0.0.0:11434"
EOF
```

Wichtig: Die erste leere `Environment=` Zeile ist notwendig, um vorherige Environment-Einträge zu löschen. Ohne diese Zeile kann der alte Wert weiterhin aktiv bleiben.

2 systemd neu laden & Ollama neu starten

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
```

3 Verifizieren

```
sudo systemctl cat ollama | grep -A5 "override.conf"
sudo systemctl show ollama -p Environment
sudo ss -lntp | grep 11434
```

Erwartete Ausgabe von `ss` :

```
LISTEN 0 4096 *:11434 *:~ users:((("ollama",pid=...,fd=4))
```

Erfolgreich wenn: `ss` zeigt `*:11434` oder `0.0.0.0:11434` statt `127.0.0.1:11434` .
Dann ist Ollama aus Docker-Containern erreichbar.

3. NVIDIA Container Toolkit installieren

Damit der Odysseus-Container die GPU des Hosts nutzen kann, muss das NVIDIA Container Toolkit installiert und Docker entsprechend konfiguriert werden.

1 Voraussetzungen prüfen

```
sudo apt-get update
sudo apt-get install -y curl gpg
```

2 NVIDIA Repository hinzufügen

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
| sudo gpg --batch --yes --dearmor \
-o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg

curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-
container-toolkit.list \
| sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-
toolkit-keyring.gpg] https://#g' \
| sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

3 Toolkit installieren

```
sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
```

4 Docker Runtime konfigurieren & neu starten

```
sudo nvidia-ctl runtime configure --runtime=docker
sudo systemctl restart docker
```

5 GPU-Passthrough testen

```
docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
```

Erfolgreich wenn: `nvidia-smi` im Container die GPU anzeigt (z.B. "NVIDIA RTX PRO 2000 Blackwell").

6 Odysseus GPU-Diagnose (optional, nach Installation)

```
cd /opt/odysseus
scripts/check-docker-gpu.sh
```

4. Odysseus – Repository klonen & Branch wählen

1 Verzeichnis anlegen & Berechtigungen setzen

```
sudo mkdir -p /opt/odysseus  
sudo chown -R "$USER":"$USER" /opt/odysseus  
cd /opt/odysseus
```

2 Repository klonen

```
git clone https://github.com/pewdiepie-archdaemon/odysseus.git .
```

3 Stablen Branch auschecken

```
git checkout main
```

Branch-Wahl:

main = stabiler Release-Branch (empfohlen für Produktion)

dev = Entwicklungs-Branch mit neuesten Features (weniger stabil)

4 Beispiel-Umgebungsdatei kopieren

```
cp .env.example .env
```

5. Umgebungsdatei (.env) konfigurieren

Die `.env` Datei steuert alle wesentlichen Einstellungen von Odysseus. Die folgende Konfiguration ist auf das stonecloud.at-Setup abgestimmt.

```
nano /opt/odysseus/.env
```

Folgende Werte eintragen / anpassen:

```
#####
# Odysseus - Environment (.env)
# Setup: stonecloud.at / Host: 172.22.222.114
#####

#####
# Network / Reverse Proxy
#####

# Bind Odysseus Web UI auf der Host-LAN-IP
# (damit NPM via 172.22.222.114:7000 proxyen kann)
APP_BIND=172.22.222.114
APP_PORT=7000

#####
# Auth & Security
#####

AUTH_ENABLED=true
LOCALHOST_BYPASS=false

# HTTPS wird am NPM terminiert -> Cookies müssen Secure sein
SECURE_COOKIES=true

# CORS Origins
ALLOWED_ORIGINS=http://localhost,http://127.0.0.1,https://
odysseus.stonecloud.at

# Admin-Benutzer (Passwort leer lassen = wird beim ersten Start auto-generiert)
ODYSSEUS_ADMIN_USER=admin
ODYSSEUS_ADMIN_PASSWORD=

#####
# Ollama (läuft am Host)
#####

# Container erreicht Host-Services via host.docker.internal
OLLAMA_BASE_URL=http://host.docker.internal:11434/v1

# Embeddings ebenfalls via Ollama
EMBEDDING_URL=http://host.docker.internal:11434/v1/embeddings
```

```

# LLM-Discovery (optional, für automatische Modell-Erkennung)
LLM_HOST=host.docker.internal
LLM_HOSTS=host.docker.internal,172.22.222.114

#####
# Web Search (SearXNG)
#####

# Externer SearXNG-Endpunkt
# Hinweis: In docker-compose.yml muss SEARXNG_INSTANCE als Variable
# definiert sein, damit dieser Wert greift (siehe Abschnitt 10)
SEARXNG_INSTANCE=https://search.stonecloud.at/

#####
# GPU (NVIDIA Blackwell)
#####

# NVIDIA Compose-Overlay aktivieren
# (erfordert NVIDIA Container Toolkit am Host)
COMPOSE_FILE=docker-compose.yml:docker/gpu.nvidia.yml

#####
# Bundled Services (loopback-only, sicher)
#####

CHROMADB_BIND=127.0.0.1
NTFY_BIND=127.0.0.1

```

Sicherheitshinweis: `APP_BIND=172.22.222.114` macht Odysseus direkt im LAN auf Port 7000 erreichbar. Das ist für den NPM-Proxy notwendig. Wer direkten LAN-Zugriff verhindern will, setzt `APP_BIND=127.0.0.1` und konfiguriert NPM so, dass es über das Docker-Netz proxyt.

6. Stack starten & GPU-Overlay aktivieren

1 Stack erstmalig bauen & starten

```
cd /opt/odysseus
docker compose up -d --build
```

Beim ersten Start werden alle Images gepullt und der Odysseus-Container gebaut (~2-5 Minuten). Folgende Container werden gestartet:

Container	Funktion	Internes Netz
odysseus-odysseus-1	Hauptanwendung (Web UI + Backend)	172.19.0.4
odysseus-chromadb-1	Vektor-Datenbank für RAG	172.19.0.2
odysseus-ntfy-1	Push-Benachrichtigungen	172.19.0.3
odysseus-searxng-1	Lokale Suchmaschine (intern)	172.19.0.x

2 Status prüfen

```
docker compose ps
```

3 GPU-Overlay verifizieren (nach dem Start)

```
cd /opt/odysseus
scripts/check-docker-gpu.sh
```

Erwartetes Ergebnis: "Results: 3 passed, 0 failed" – GPU-Passthrough funktioniert.

4 Bekanntes Problem: Port 8080 bereits belegt

Falls beim ersten Start der Fehler `failed to bind host port 127.0.0.1:8080/tcp: address already in use` erscheint: Ein anderer Dienst (z.B. ein weiterer SearXNG oder Webserver) belegt Port 8080. Lösung: Den anderen Dienst stoppen oder in der `docker-compose.yml` den SearXNG-Port ändern, dann erneut `docker compose up -d`.

7. Erster Login & Admin-Passwort

1 Temporäres Passwort aus den Logs holen

```
cd /opt/odysseus
docker compose logs odysseus | grep -i "Temporary password"
```


Ausgabe (Beispiel):

```
odysseus-1 | [ok] Initial admin user created (admin)
odysseus-1 | Temporary password: 7FtUU00f8MqPwG1a6AFq6Zr3
```

2 Login

URL: `https://odysseus.stonecloud.at` (nach NPM-Einrichtung, Abschnitt 8)

Username: `admin`

Password: Das temporäre Passwort aus den Logs

Wichtig: Da `SECURE_COOKIES=true` gesetzt ist, funktioniert der Login **nur über HTTPS**. Der direkte HTTP-Zugriff auf `http://172.22.222.114:7000` schlägt beim Login fehl. Zuerst NPM einrichten (Abschnitt 8), dann über die HTTPS-Domain einloggen.

3 Passwort nach erstem Login ändern

Nach dem ersten Login unter **Settings** → **Account** das temporäre Passwort durch ein sicheres Passwort ersetzen.

8. Nginx Proxy Manager – Reverse Proxy einrichten

NPM übernimmt die SSL-Terminierung und leitet Anfragen an den Odysseus-Container weiter.

1 Neuen Proxy Host anlegen

Im NPM-Dashboard: **Proxy Hosts** → **Add Proxy Host**

Feld	Wert
Domain Names	odysseus.stonecloud.at
Scheme	http
Forward Hostname / IP	172.22.222.114
Forward Port	7000
Websockets Support	aktiviert (Pflicht für Chat-Funktionen)
Block Common Exploits	aktiviert (empfohlen)

2 SSL-Zertifikat einrichten

Im Tab **SSL**:

Feld	Wert
SSL Certificate	Let's Encrypt (neu anfordern) oder bestehendes Zertifikat
Force SSL	aktiviert
HTTP/2 Support	aktiviert (empfohlen)
HSTS Enabled	optional

3 Custom Nginx Config (Advanced Tab)

Für optimale WebSocket-Unterstützung im Advanced-Tab einfügen:

```
proxy_read_timeout 300s;
proxy_send_timeout 300s;
proxy_buffering off;
```

4 Speichern & testen

Nach dem Speichern: <https://odysseus.stonecloud.at> im Browser aufrufen. Der Login-Screen von Odysseus sollte erscheinen.

Warum Forward auf 172.22.222.114 und nicht auf Container-Name?

NPM läuft nicht im selben Docker-Netz wie Odysseus (`odysseus_default`). Daher wird über die Host-LAN-IP weitergeleitet. Odysseus ist via `APP_BIND=172.22.222.114` auf dieser IP erreichbar.

9. Ollama in Odysseus einbinden

Nachdem Ollama korrekt auf `0.0.0.0:11434` lauscht (Abschnitt 2), kann es in Odysseus als Modell-Endpoint eingetragen werden.

1 Verbindung aus dem Container testen (optional)

```
docker run --rm --network odysseus_default curlimages/curl:8.8.0 \
-sS http://host.docker.internal:11434/api/tags | head
```

Wenn Modelle aufgelistet werden, ist die Verbindung korrekt.

2 Endpoint in Odysseus eintragen

In Odysseus: **Settings** → **Add Models** → **Add Local Models**

Feld	Wert
Typ	LLM
Endpoint URL	<code>http://host.docker.internal:11434/v1</code>

Auf **Add** klicken. Erwartete Rückmeldung:

```
Online – found 13 models: gemma4:latest, gemma4:26b, qwen3:8b, ...
```

Verfügbare Modelle (Stand Installation):

gemma4:latest (8B), gemma4:26b (26B), qwen3:8b, glm-4.7-flash (30B), gpt-oss:20b, gemma3:12b, mistral-small3.2 (24B), translategemma:12b, lfm2.5:8b, ministral-3:14b, minicpm-v4.5, minicpm-v4.6, granite4.1:8b

3 Standard-Modell festlegen

In Odysseus: **Settings** → **AI Defaults** → gewünschtes Modell als Standard auswählen.

10. SearXNG (extern) einbinden

Odysseus bringt einen eigenen SearXNG-Container mit. Alternativ kann der externe SearXNG unter `https://search.stonecloud.at/` genutzt werden.

Option A: Über die Odysseus-UI (einfachste Methode)

In Odysseus: **Settings** → **Search** → SearXNG-URL auf `https://search.stonecloud.at/` setzen und speichern.

Option B: Über .env + docker-compose.yml (dauerhaft)

Damit der Wert aus der `.env` greift, muss in `/opt/odysseus/docker-compose.yml` beim `odysseus`-Service die Zeile:

```
- SEARXNG_INSTANCE=http://searxng:8080
```

geändert werden zu:

```
- SEARXNG_INSTANCE=${SEARXNG_INSTANCE:-http://searxng:8080}
```

Danach:

```
cd /opt/odysseus  
docker compose up -d
```

11. Portainer – Stack-Verwaltung

Der Odysseus-Stack wurde via CLI (`docker compose up`) gestartet. Portainer erkennt ihn automatisch, hat aber eingeschränkte Kontrolle ("Limited Control").

Hinweis: "This stack was created outside of Portainer. Control over this stack is limited." – Das ist normal und kein Fehler. Grundlegende Aktionen (Start, Stop, Restart einzelner Container) sind weiterhin über Portainer möglich.

Empfohlener Workflow für Updates

```
cd /opt/odysseus
git pull
docker compose up -d --build
```

Stack komplett neu starten

```
cd /opt/odysseus
docker compose down
docker compose up -d --build
```

Logs einsehen

```
# Alle Container
docker compose logs --tail=100

# Nur Odysseus
docker compose logs --tail=100 odysseus

# Live-Follow
docker compose logs -f odysseus
```

12. Referenz: Wichtige Ports & Netzwerke

Dienst	Port (Host)	Port (intern)	Bind
Odysseus Web UI	7000	7000	172.22.222.114
ChromaDB	8100	8000	127.0.0.1
ntfy	8091	80	127.0.0.1
SearXNG (intern)	8080*	8080	127.0.0.1
Ollama (Host)	11434	–	0.0.0.0
Portainer	9443 (HTTPS)	–	0.0.0.0

* SearXNG Port 8080 kann mit anderen Diensten kollidieren. Bei Konflikt in docker-compose.yml anpassen.

Docker-Netzwerk	Subnetz	Verwendung
bridge (default)	172.17.0.0/16	Standard Docker Bridge
odysseus_default	172.19.0.0/16	Internes Odysseus-Netz (alle Odysseus-Container)
1panel-network	172.18.0.0/16	1Panel Stack
host.docker.internal	–	Alias für Host-Gateway (172.22.222.114)

Wichtige Dateipfade

Pfad	Inhalt
/opt/odysseus/.env	Hauptkonfiguration
/opt/odysseus/docker-compose.yml	Stack-Definition
/opt/odysseus/data/	Datenbank, Uploads, Sessions, Cache
/opt/odysseus/logs/	Anwendungs-Logs
/etc/systemd/system/ollama.service.d/override.conf	Ollama systemd Override (OLLAMA_HOST)